



ŽILINSKÁ UNIVERZITA V ŽILINE

Fakulta riadenia
a informatiky

Semestrálna práca z predmetu
vývoj aplikácií pre mobilné zariadenia

MOBILNÁ APLIKÁCIA NA SLEDOVANIE SILOVÉHO TRÉNINGU

Vypracoval: Filip Gazdík

Študijná skupina: 5ZYR22

Akademický rok: 2025/2026

V Žiline dňa 23.5.2026



Obsah

Úvod	2
Prehľad podobných aplikácií	2
Analýza navrhovanej aplikácie	7
UML Diagram prípadov použitia	9
Návrh architektúry aplikácie	10
UML Diagram tried.....	10
Skutočný návrh riešenia problému	14

Úvod

Cieľom tejto semestrálnej práce bol návrh a implementácia mobilnej aplikácie zameranej na sledovanie silového tréningu a progresu používateľa. Aplikácia bola inšpirovaná existujúcimi riešeniami, ako sú napríklad Hevy alebo Liftoff, pričom jej hlavným cieľom bolo ponúknuť jednoduché, prehľadné a efektívne rozhranie pre zaznamenávanie tréningov.

Motiváciou pre vytvorenie tejto aplikácie je rastúca popularita fitness a potreba systematického sledovania výkonu. Mnoho používateľov si vedie tréningové záznamy neprehľadným spôsobom alebo využíva aplikácie, ktoré sú príliš komplexné alebo obsahujú zbytočné funkcie. Vytvorená aplikácia sa preto zameriava na kombináciu jednoduchosti, rýchlosti a prehľadnosti, pričom zároveň poskytuje dostatočné množstvo funkcionality na efektívne sledovanie progresu.

Aplikácia umožňuje používateľovi vytvárať a ukladať tréningové rutiny, pridávať jednotlivé cviky a zaznamenávať série vrátane počtu opakovaní a použitej váhy. Súčasťou aplikácie je história tréningov, štatistiky s grafmi progresu, odpočinkový časovač, widget na ploche zariadenia a automatické pripomienky tréningu. Dôraz bol kladený na intuitívne používateľské rozhranie a rýchlu interakciu, aby bolo možné zaznamenávať tréning priamo počas cvičenia bez zbytočného zdržania.

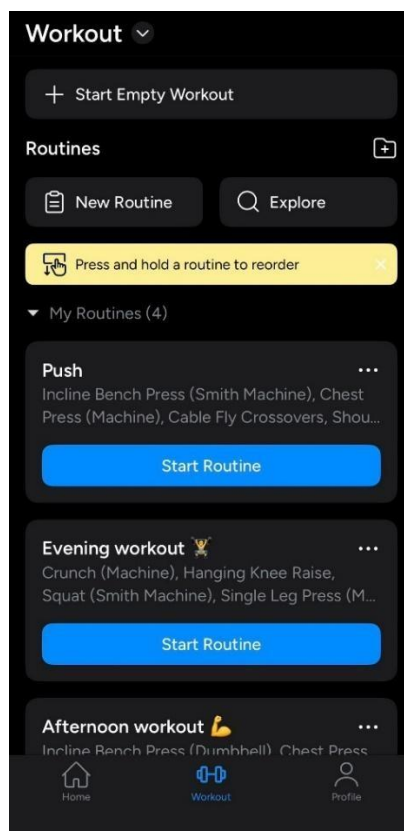
Z technologického hľadiska bola aplikácia implementovaná v jazyku Kotlin s využitím moderného UI toolkit-u Jetpack Compose. Pri implementácii bola dodržaná odporúčaná architektúra MVVM (Model–View–ViewModel), najmä princípy oddelenia zodpovedností a riadenia používateľského rozhrania pomocou dátového modelu. Na správu stavu aplikácie bol využitý komponent ViewModel v kombinácii so StateFlow, na ukladanie dát lokálna databáza Room a na prácu na pozadí knižnica WorkManager.

Výsledkom práce je funkčná aplikácia, ktorá spĺňa požiadavky na modernú mobilnú aplikáciu z hľadiska architektúry, použiteľnosti a kvality implementácie, a zároveň poskytuje používateľovi praktický nástroj na každodenné sledovanie tréningového progresu.

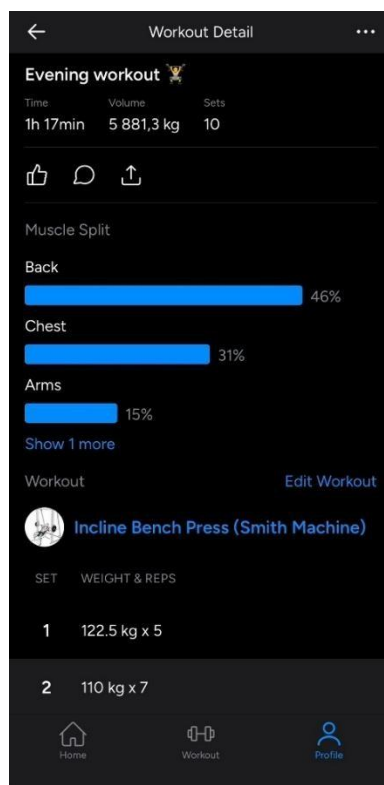
Prehľad podobných aplikácií

V súčasnosti existuje množstvo mobilných aplikácií zameraných na sledovanie tréningov a fitness progresu. V tejto kapitole sú analyzované tri vybrané aplikácie, ktoré riešia podobný problém ako navrhovaná aplikácia.

Hevy



Obr. 1: Domovská obrazovka aplikácie Hevy



Obr. 2: Obrazovka tréningu aplikácie Hevy

Aplikácia Hevy patrí medzi najpopulárnejšie nástroje na sledovanie silového tréningu. Umožňuje používateľovi vytvárať tréningové rutiny, zaznamenávať jednotlivé cviky a série a sledovať svoj progres v čase.

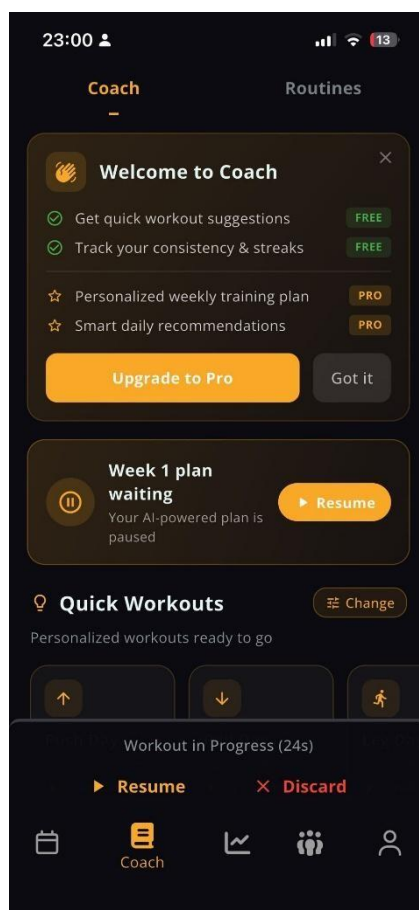
Výhody:

- prehľadné a moderné používateľské rozhranie
- rýchle zaznamenávanie sérií počas tréningu
- podpora tréningových plánov (routines)
- detailné štatistiky (objem, čas, počet sérií)

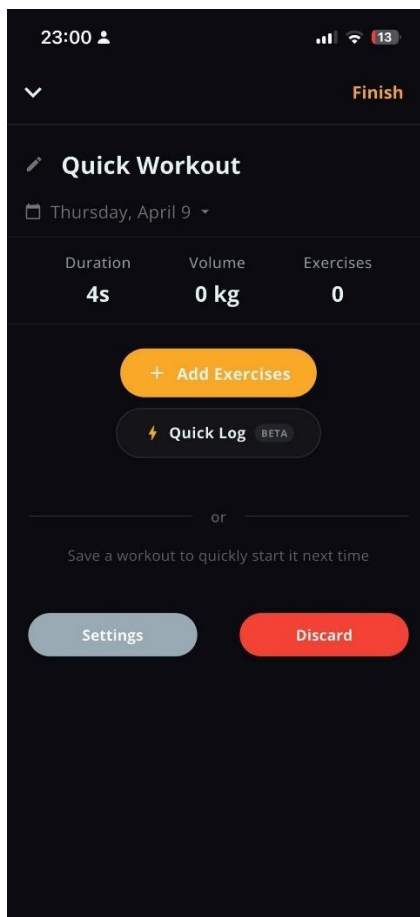
Nevýhody:

- niektoré pokročilé funkcie sú dostupné len v platenej verzii
- obmedzené inteligentné odporúčania pre tréning

Stronger



Obr. 3: Domovská obrazovka aplikácie Stronger



Obr. 4: Obrazovka tréningu aplikácie Stronger

Aplikácia Strong (často označovaná aj ako Stronger) je populárny nástroj na sledovanie silového tréningu, ktorý sa zameriava na jednoduchosť a efektívnosť pri zaznamenávaní tréningov.

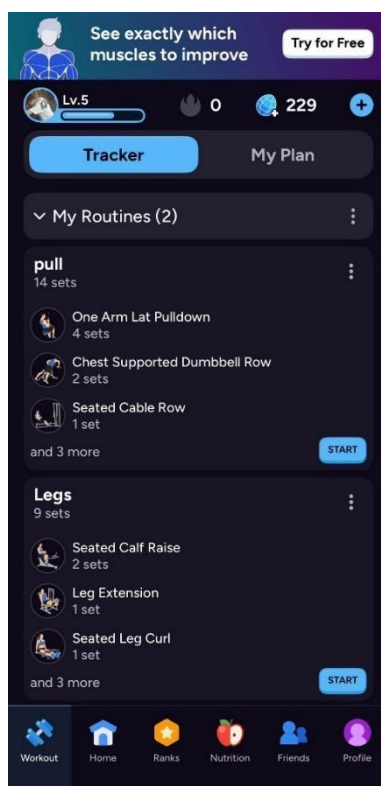
Výhody:

- veľmi jednoduché a intuitívne používateľské rozhranie
- rýchle zaznamenávanie cvikov a sérií počas tréningu
- prehľadná história tréningov
- vhodná pre začiatočníkov aj pokročilých používateľov

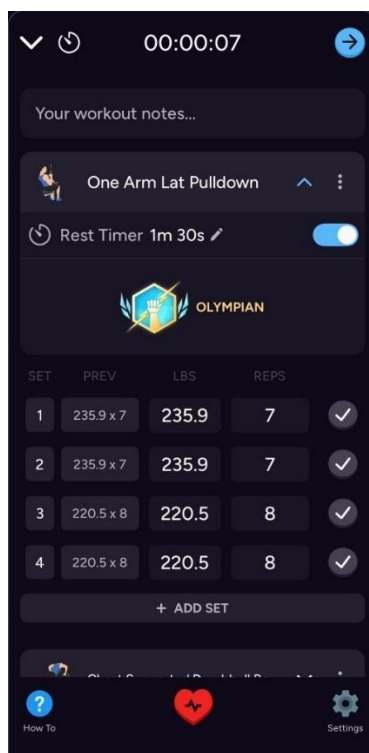
Nevýhody:

- menej pokročilé analytické a štatistické funkcie
- obmedzené možnosti personalizácie tréningových plánov
- jednoduchší dizajn v porovnaní s modernejšími aplikáciami

Liftoff



Obr. 5: Domovská obrazovka aplikácie Liftoff



Obr. 6: Obrazovka tréningu aplikácie Liftoff



Aplikácia Liftoff je moderná fitness aplikácia zameraná na sledovanie tréningov s dôrazom na vizuálny dizajn a gamifikáciu. Okrem základného zaznamenávania tréningov ponúka aj prvky motivácie, ako sú úrovne, progres a odporúčania.

Výhody:

- atraktívne a moderné používateľské rozhranie
- prvky gamifikácie (levely, progres, motivácia)
- prehľadné zobrazenie tréningových plánov
- možnosť sledovania pokroku v rôznych oblastiach

Nevýhody:

- vyššia komplexnosť rozhrania
- niektoré funkcie môžu byť pre začiatočníkov neprehľadné
- dôraz na dizajn môže ísť na úkor rýchlosti používania

Analyzované aplikácie ponúkajú rôzne prístupy k riešeniu sledovania tréningu. Aplikácia Hevy poskytuje vyvážený pomer medzi jednoduchosťou a funkcionalitou, Strong sa zameriava na maximálnu jednoduchosť a rýchlosť používania, zatiaľ čo Liftoff kladie dôraz na vizuálny dizajn a motiváciu používateľa prostredníctvom gamifikácie.

Navrhovaná aplikácia kombinuje výhody týchto riešení. Hlavný dôraz bol kladený na:

- rýchle a jednoduché zaznamenávanie tréningov (inšpirácia Hevy)
- intuitívne ovládanie (inšpirácia Strong)
- moderný dizajn a motivačné prvky (inšpirácia Liftoff)

Cieľom je vytvoriť aplikáciu, ktorá je efektívna na každodenné používanie počas tréningu, pričom si zachová prehľadnosť a zároveň ponúkne moderný používateľský zážitok.

Analýza navrhovanej aplikácie

Navrhovaná aplikácia je mobilná fitness aplikácia určená na zaznamenávanie silových tréningov, vytváranie tréningových rutín a sledovanie progresu používateľa v čase. Jej hlavným cieľom je umožniť používateľovi jednoducho a rýchlo zapisovať priebeh tréningu, evidovať jednotlivé cviky a série, a zároveň poskytovať prehľad o histórii tréningov a základných štatistikách.

Aplikácia je navrhnutá predovšetkým pre jednotlivcov, ktorí sa venujú silovému tréningu vo fitness centre alebo doma a chcú mať svoje tréningové dáta uložené v prehľadnej digitálnej forme. Dôraz je kladený najmä na jednoduchosť používania počas cvičenia, aby používateľ vedel zaznamenávať výsledky bez zbytočných krokov a zdržania.

Používateľské roly

V navrhovanej aplikácii sa predpokladá jedna hlavná používateľská rola:

Bežný používateľ

Bežný používateľ je osoba, ktorá aplikáciu používa na vytváranie tréningových plánov, zaznamenávanie tréningov a sledovanie svojho progresu.

Jeho hlavné oprávnenia a možnosti sú:

- vytvárať nové tréningové rutiny,
- upravovať a mazať existujúce rutiny,
- pridávať cviky do tréningu,
- zapisovať série, opakovania a použitú váhu,
- spúšťať tréning podľa vytvorenej rutiny,
- prezerať históriu absolvovaných tréningov,
- sledovať základné štatistiky a progres.

V rámci počiatočnej verzie aplikácie sa nepredpokladá administrátor ani viacero používateľských rolí, keďže aplikácia je navrhnutá ako osobný nástroj pre jedného používateľa.

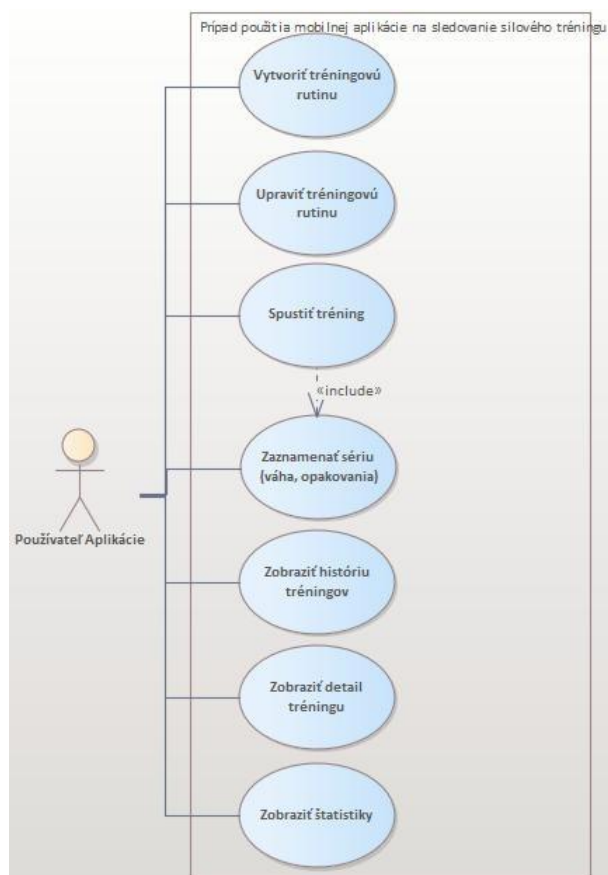
Prehľad hlavných funkcií aplikácie

Z pohľadu používateľa aplikácia obsahuje najmä tieto funkčné celky:

- správa tréningových rutín,
- evidencia aktívneho tréningu,
- evidencia cvikov a sérií,
- história tréningov,
- detail tréningu,
- štatistiky a progres,
- pomocné funkcie, ako napríklad časovač odpočinku.

UML Diagram prípadov použitia

Hlavné funkcie aplikácie a ich vzťahy sú znázornené na Obr. 7.



Obr. 7: Diagram prípadov použitia tvorenej aplikácie

Mimofunkčné požiadavky

Okrem funkčných požiadaviek je potrebné definovať aj mimofunkčné požiadavky aplikácie.

Použiteľnosť

Aplikácia musí byť navrhnutá tak, aby bola intuitívna a jednoducho ovládateľná aj počas cvičenia. Používateľ musí vedieť rýchlo pridávať série a pohybovať sa medzi jednotlivými obrazovkami bez zbytočných krokov.

Výkon

Aplikácia musí reagovať plynulo a bez výrazných oneskorení. Ukladanie tréningových dát a načítavanie histórie musí prebiehať rýchlo aj pri väčšom počte záznamov.

Spoľahlivosť

Aplikácia musí správne uchovávať používateľské dáta a nesmie dochádzať k ich strate pri bežnom používaní. Dôležité je aj korektné správanie pri zmene orientácie zariadenia.

Kompatibilita

Aplikácia musí byť funkčná na bežných Android zariadeniach a musí byť navrhnutá v súlade s požiadavkami predmetu, teda v jazyku Kotlin s použitím Jetpack Compose.

Offline dostupnosť

Základné funkcionality aplikácie by mali byť dostupné aj bez internetového pripojenia, najmä vytváranie rutín, zaznamenávanie tréningov a prezeranie histórie. To je dôležité pre používateľov, ktorí cvičia v prostredí bez stabilného internetu.

Prehľadnosť a konzistentnosť rozhrania

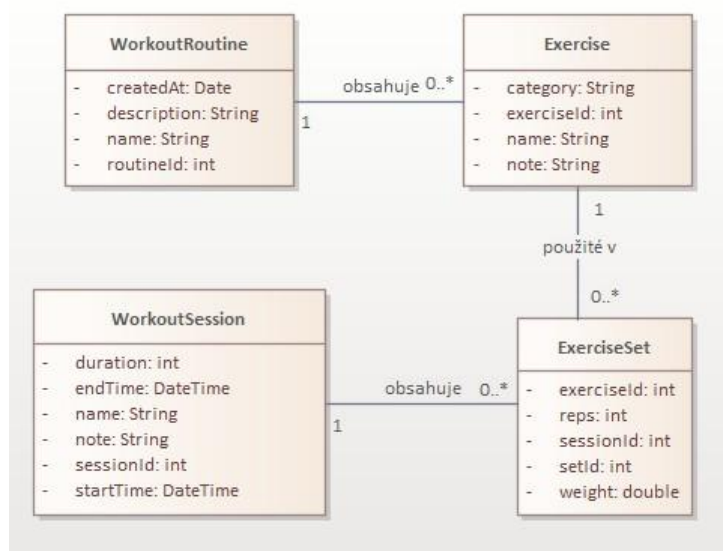
Všetky obrazovky aplikácie by mali používať jednotný vizuálny štýl, konzistentné navigačné prvky a rovnaký spôsob interakcie s formulármi a tlačidlami.

Návrh architektúry aplikácie

Aplikácia pracuje s dátami používateľa, ktoré sú ukladané lokálne v zariadení. Keďže ide o aplikáciu na sledovanie silového tréningu, základom architektúry je práca s entitami reprezentujúcimi tréningové rutiny, jednotlivé cviky, tréningové záznamy a série. Architektúra aplikácie bola navrhnutá tak, aby zabezpečovala prehľadné oddelenie používateľského rozhrania, aplikačnej logiky a dátovej vrstvy.

UML Diagram tried

Dátový model aplikácie je znázornený na Obr. 8.



Obr. 8: Diagram tried tvorenej aplikácie

Diagram tried znázorňuje základnú štruktúru dátového modelu aplikácie, vrátane entít, ich atribútov a vzťahov medzi nimi.

Dátový model aplikácie

Základ dátového modelu tvoria nasledujúce entity:

WorkoutRoutine

Entita WorkoutRoutine reprezentuje tréningovú rutinu vytvorenú používateľom. Obsahuje základné údaje, ako napríklad:

- identifikátor rutiny,
- názov rutiny,
- popis rutiny,
- dátum vytvorenia.

Jedna tréningová rutina môže obsahovať viacero cvikov.

Exercise

Entita Exercise reprezentuje konkrétny cvik zaradený do rutiny alebo tréningu. Obsahuje:

- identifikátor cviku,
- názov cviku,
- kategóriu cviku,

Jeden cvik sa môže vyskytovať vo viacerých tréningových rutinách alebo tréningoch.

WorkoutSession

Entita WorkoutSession reprezentuje konkrétne absolvovaný tréning. Obsahuje:

- identifikátor tréningu,
- názov tréningu,
- dátum a čas začiatku tréningu,
- dátum a čas ukončenia tréningu,
- celkové trvanie tréningu,
- voliteľnú poznámku.

Jeden absolvovaný tréning obsahuje viacero vykonaných cvikov a sérií.

ExerciseSet

Entita ExerciseSet reprezentuje jednu konkrétnu sériu vykonanú pri cviku. Obsahuje:

- identifikátor série,
- identifikátor tréningu,
- identifikátor cviku,
- poradové číslo série,

- počet opakovaní,
- použitú váhu.

Táto entita je dôležitá pre ukladanie detailných údajov o výkone používateľa a pre následné štatistiky.

Vzťahy medzi entitami

Medzi uvedenými entitami existujú nasledujúce vzťahy:

- jedna **WorkoutRoutine** obsahuje viacero **Exercise**,
- jedna **WorkoutSession** obsahuje viacero **ExerciseSet**,
- jeden **Exercise** môže byť použitý vo viacerých rutinách a tréningoch.

Väzba medzi rutinou a cvikmi je implementovaná pomocou prepájacej tabuľky `RoutineExerciseCrossRef`, ktorá rieši vzťah M:N a zároveň uchováva poradie cvikov v rutine.

Spracovanie dát v aplikácii

Aplikácia je navrhnutá ako offline-first riešenie, čo znamená, že všetky dáta sú ukladané do lokálnej databázy zariadenia. Používateľ môže aplikáciu používať aj bez internetového pripojenia. Na ukladanie a načítavanie dát je použitá lokálna databáza `Room`, ktorá zabezpečuje efektívny a bezpečný prístup k údajom.

Tok informácií v aplikácii prebieha nasledovne:

- používateľ vykoná akciu v používateľskom rozhraní,
- požiadavka sa odošle do `ViewModelu`,
- `ViewModel` spracuje logiku a komunikuje s dátovou vrstvou,
- dátová vrstva uloží alebo načíta údaje z databázy,
- výsledok sa vráti späť do `ViewModelu`,
- používateľské rozhranie sa aktualizuje na základe nového stavu.

Koncepčný návrh aplikácie

Pri implementácii bola použitá odporúčaná viacvrstvová architektúra rozdelená na nasledujúce časti:

Prezentačná vrstva (UI layer)

Táto vrstva je zodpovedná za zobrazovanie údajov používateľovi a za spracovanie jeho interakcií. Je implementovaná pomocou `Jetpack Compose`. Každá obrazovka reprezentuje samostatnú funkčnosť aplikácie — domovskú obrazovku, aktívny tréning, históriu, detail tréningu alebo štatistiky.

Vrstva aplikačnej logiky

Táto vrstva je reprezentovaná triedami ViewModel. ViewModel uchováva stav obrazovky, spracúva používateľské akcie a komunikuje s dátovou vrstvou. Stav aplikácie je oddelený od samotného používateľského rozhrania a je odolný voči zmenám konfigurácie zariadenia (napríklad rotácia displeja).

Dátová vrstva

Dátová vrstva zabezpečuje prístup k lokálnym dátam. Pozostáva z databázových entít, DAO rozhraní a Repository vrstvy. Repository slúži ako sprostredkovateľ medzi ViewModelom a databázou, čím oddeľuje logiku práce s dátami od zvyšku aplikácie.

Navrhovaná architektúra aplikácie

Aplikácia využíva architektúru **MVVM (Model–View–ViewModel)**, ktorá je odporúčaná pre moderné Android aplikácie a umožňuje jasné oddelenie používateľského rozhrania od logiky a dát.

- **Model** - reprezentovaný dátovými triedami, databázou Room a Repository vrstvou. Zodpovedá za uchovávanie a spracovanie dát aplikácie.
- **View** - tvorený composable funkciami v Jetpack Compose. Zobrazuje aktuálny stav aplikácie a odosiela používateľské akcie do ViewModelu. ViewModel
- **ViewModel** - sprostredkúva komunikáciu medzi View a Modelom. Uchováva stav obrazoviek a poskytuje UI vrstve pripravené dáta prostredníctvom StateFlow.

Táto architektúra prináša nasledujúce výhody:

- oddelenie logiky od používateľského rozhrania,
- jednoduchšiu testovateľnosť,
- lepšiu udržiavateľnosť kódu,
- vyššiu prehľadnosť projektu,
- odolnosť voči zmenám konfigurácie zariadenia.

Možné diagramy a procesy

Súčasťou návrhu architektúry je jednoduchý diagram aktivít alebo sekvenčný diagram. Napríklad proces zaznamenania série prebieha nasledovne:

- používateľ otvorí aktívny tréning,
- zadá počet opakovaní a váhu,
- klikne na potvrdenie,
- údaje sa odošlú do ViewModelu,
- ViewModel požiada repository o uloženie série,
- repository zapíše údaje do databázy,



- používateľské rozhranie sa aktualizuje.

Podobne je možné znázorniť aj proces vytvorenia tréningovej rutiny alebo zobrazenia histórie tréningov.

Skutočný návrh riešenia problému

Moderný životný štýl si vyžaduje systematický prístup k fyzickej aktivite. Aplikácia Workout Tracker rieši problém sledovania a plánovania silového tréningu priamo na mobilnom zariadení. Používateľ si môže vytvoriť vlastné tréningové rutiny (napr. Push / Pull / Legs split), počas tréningu zaznamenávať série s počtom opakovaní a použitou váhou, prezerať históriu dokončených tréningov a sledovať progresiu v stĺpcových grafoch. Systém automaticky upozorní používateľa notifikáciou, ak netrénoval viac ako tri dni, a zobrazuje týždenný počet tréningov priamo na ploche zariadenia prostredníctvom widgetu.

Diagram prípadov použitia – Tabuľka

ID	Názov	Popis	Aktér
UC1	Prezeranie rutín	Používateľ vidí zoznam tréningových rutín na hlavnej obrazovke.	Používateľ
UC2	Vytvorenie / úprava rutiny	Používateľ vytvorí novú rutinu alebo upraví existujúcu – pridá cviky, zmení poradie.	Používateľ
UC3	Spustenie tréningu	Používateľ spustí tréning na základe vybranej rutiny alebo voľný tréning.	Používateľ
UC4	Zaznamenanie série	Počas tréningu používateľ zadá počet opakovaní a váhu, systém uloží sériu do databázy.	Používateľ
UC5	Dokončenie tréningu	Používateľ ukončí tréning; systém uloží čas, dĺžku a celkový objem, odošle notifikáciu.	Používateľ
UC6	Prezeranie histórie tréningov	Používateľ prezerá zoznam všetkých dokončených tréningov zoradených od najnovšieho.	Používateľ
UC7	Prezeranie štatistík	Používateľ vidí súhrnné štatistiky a grafy – dĺžky tréningov, série, progres váhy.	Používateľ
UC8	Výber cviku pre graf progresu váhy	Používateľ vyberie cvik z dropdownu; systém zobrazí graf maximálnej váhy za posledné sessions.	Používateľ
UC9	Pokračovanie v prerušenom tréningu	Po reštarte aplikácie systém deteguje nedokončenú session a ponúkne pokračovanie.	Používateľ
UC10	Odoslanie pripomienky	WorkManager skontroluje dátum posledného tréningu; ak > 3 dni, odošle notifikáciu.	Systém (WorkManager)
UC11	Aktualizácia widgetu	Po každom spustení workera sa aktualizuje počítadlo tréningov na widgete.	Systém (WorkManager)

Návrh riešenia

Aplikácia je postavená na architektúre MVVM (Model – View – ViewModel) doplnenej o Repository pattern. Táto architektúra zabezpečuje striktné oddelenie používateľského rozhrania od obchodnej logiky, čo zvyšuje testovateľnosť a udržiateľnosť kódu.



Vrstvy aplikácie

UI VRSTVA – JETPACK COMPOSE OBRAZOVKY; zbiera data z ViewModelu cez StateFlow

VIEWMODEL VRSTVA – spravuje stav obrazovky; spúšťa korutiny volá Repository

REPOSITORY VRSTVA – jediný zdroj pravdy; skrýva implementáciu DAO pred ViewModelmi

DÁTOVÁ VRSTVA – Room databáza, Flow dotazy

UI (COMPOSE) <-> ViewModel <-> Repository <-> DAO <-> ROOM

ViewModel prežíva rotáciu displeja – stav (StateFlow) zostáva zachovaný bez nutnosti opätovného načítania dát. Room pri zmene dát automaticky invaliduje Flow, čím sa spúšťa rekonpozícia UI bez manuálneho obnovenia.

Diagram tried

Trieda	Tabuľka	Kľúčové atribúty	Poznámka
WorkoutRoutine	workout_routines	id, name, description, createdAt	Tréningová rutina
Exercise	exercises	id, name, category	Zdieľaný cvik
RoutineExerciseCrossRef	routine_exercise_cross_ref	routineId (FK), exerciseId (FK), orderIndex	Junction tabuľka M:N
WorkoutSession	workout_sessions	id, name, routineId, startTime, endTime, durationSeconds, notes	endTime=null počas tréningu
ExerciseSet	exercise_sets	id, sessionId, exerciseId, exerciseName, setNumber, reps, weightKg	exerciseName denormalizovaný

Tabuľka 1 - diagram tried



DAO rozhrania

DAO trieda	Spravuje	Kľúčové metódy
WorkoutRoutineDao	WorkoutRoutine, RoutineExerciseCrossRef	getAllRoutines():Flow, insertRoutine(), insertCrossRef(), updateExerciseOrder()
ExerciseDao	Exercise	getAllExercises():Flow, getExercisesForRoutine():Flow, insertExercise()
WorkoutSessionDao	WorkoutSession	getPagedSessionSummaries():PagingSource, getSessionsSince():Flow, getSessionCountSince()
ExerciseSetDao	ExerciseSet	getSetsForSession():Flow, getLastSessionSets(), getWeightProgressionForExercise()

Tabuľka 2 - DAO rozhrania

ViewModely

ViewModel	Obrazovka	Kľúčový stav / metódy
HomeViewModel	HomeScreen	allRoutines:Flow, resumeSessionId, deleteRoutine()
ActiveWorkoutViewModel	ActiveWorkoutScreen	exercises:StateFlow, restTimerSeconds, addSet(), finishWorkout(), startRestTimer()
RoutineDetailViewModel	RoutineDetailScreen	routine:StateFlow, exercises (flatMapLatest), saveRoutine(), moveExerciseUp/Down()
HistoryViewModel	HistoryScreen	sessions:Flow<PagingData>, deleteSession()
StatsViewModel	StatsScreen	chartType:StateFlow, weightProgress, toggleChartType(), selectExercise()
WorkoutDetailViewModel	WorkoutDetailScreen	session:StateFlow, sets:StateFlow

Tabuľka 3 - ViewModely

Popis implementácie

Dátová vrstva

Dátová vrstva aplikácie je postavená na knižnici Room, ktorá slúži ako ORM nad SQLite databázou. Dáta sú uložené v piatich entitách. Entita **WorkoutSession** reprezentuje jeden absolvovaný tréning, obsahuje čas začiatku, konca, trvanie a voliteľnú poznámku. Pole `endTime = null` počas aktívneho tréningu signalizuje nedokončenú session. Podobne sú definované entity **WorkoutRoutine**, **Exercise**, **ExerciseSet** a prepojujúca tabuľka **RoutineExerciseCrossRef**, ktorá rieši vzťah M:N medzi rutinami a cvikmi.

```
package com.example.gazdik_vamz.data.local.entity

import ...

/** Reprezentovanie jednej tréningovej session-y. */
@Filip Gazdik
@Entity(tableName = "workout_sessions")
data class WorkoutSession(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val name: String,
    val routineId: Long? = null,
    val startTime: Long = System.currentTimeMillis(),
    val endTime: Long? = null,
    val durationSeconds: Long? = null,
    val notes: String = ""
)
```

Obr. 9: Entita *WorkoutSession*

DAO rozhrania

Pre prístup k databáze som použil **DAO** (Data Access Object) rozhrania. Každá entita má svoje vlastné DAO, kde sú definované všetky potrebné databázové operácie pomocou anotácií **@Query**, **@Insert**, **@Update** a **@Delete**. Väčšina metód vracia typ `Flow<T>`, vďaka čomu Room automaticky informuje UI o každej zmene v databáze bez toho, aby som musel dáta obnovovať manuálne. V `WorkoutSessionDao` som navyše použil `PagingSource` na stránkované načítavanie histórie tréningov, aby aplikácia zostala plynulá aj pri väčšom množstve záznamov.

```
1 package com.example.gazdik_vamz.data.local.dao
2
3 > import ...
4
5
6
7
8
9
10 /** Data Access Objects pre CREATE, READ, UPDATE a DELETE pre tabuľku workout_sessions. */
11  * Filip Gazdik
12 @Dao
13 interface WorkoutSessionDao {
14
15     /** Vrátene všetkých tréningov zoradených od najnovšieho */
16     * Filip Gazdik
17     @Query("SELECT * FROM workout_sessions ORDER BY startTime DESC")
18     fun getAllSessions(): Flow<List<WorkoutSession>>
19
20     /** Vrátene session-y s daným ID */
21     * Filip Gazdik
22     @Query("SELECT * FROM workout_sessions WHERE id = :id")
23     suspend fun getSessionById(id: Long): WorkoutSession?
24
25     /** Vloženie novej session-y a vrátenie jej ID */
26     * Filip Gazdik
27     @Insert(onConflict = OnConflictStrategy.REPLACE)
28     suspend fun insertSession(session: WorkoutSession): Long
29
30     /** Aktualizovanie prebiehajúcej session-y */
31     * Filip Gazdik
32     @Update
33     suspend fun updateSession(session: WorkoutSession)
34
35     /** Vymazanie session-y z databázy. */
36     * Filip Gazdik
37     @Delete
38     suspend fun deleteSession(session: WorkoutSession)
```

Obr. 10: DAO rozhranie WorkoutSessionDao

AppDatabase a Repository

Trieda **AppDatabase** združuje všetky entity a poskytuje prístup k jednotlivým DAO rozhraniam. Je anotovaná pomocou **@Database**, kde som uviedol zoznam všetkých entít a verziu databázy. Inštancia databázy je vytváraná ako singleton, použil som vzor companion object s **@Volatile** premennou, čo zabezpečuje, že v celej aplikácii existuje vždy len jedna inštancia databázy.

Repository vrstva slúži ako prostredník medzi **ViewModeli** a databázou. ViewModely nekomunikujú priamo s DAO, ale vždy cez Repository. Vďaka tomu je kód prehľadnejší a prípadná zmena zdroja dát by sa dotkla len Repository, nie celej aplikácie.



```
1 package com.example.gazdik_vamz.data.local
2
3 > import ...
4
5
6
7
8
9
10 /** Hlavná databáza aplikácie. */
11  * Filip Gazdik
12 @Database(
13     entities = [
14         WorkoutRoutine::class,
15         Exercise::class,
16         RoutineExerciseCrossRef::class,
17         WorkoutSession::class,
18         ExerciseSet::class
19     ],
20     version = 1,
21     exportSchema = false
22 )
23
24 @abstract class AppDatabase : RoomDatabase() {
25
26     * Filip Gazdik
27     abstract fun workoutRoutineDao(): WorkoutRoutineDao
28     * Filip Gazdik
29     abstract fun exerciseDao(): ExerciseDao
30     * Filip Gazdik
31     abstract fun workoutSessionDao(): WorkoutSessionDao
32     * Filip Gazdik
33     abstract fun exerciseSetDao(): ExerciseSetDao
34
35     * Filip Gazdik
36     companion object {
37         @Volatile
38         private var INSTANCE: AppDatabase? = null
39     }
40 }
```

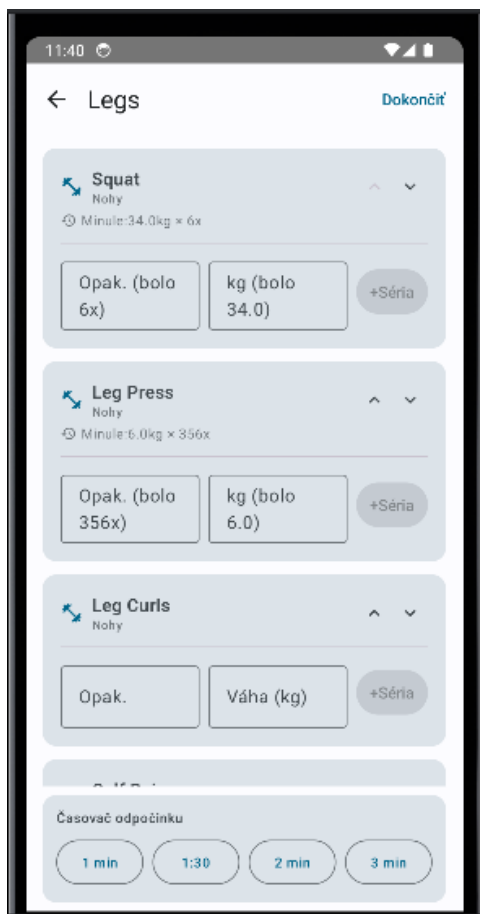
Obr. 11: Trieda AppDatabase

Viewmodel

Pre správu stavu každej obrazovky som použil triedu ViewModel. Stav je uložený v StateFlow premenných, ktoré UI sleduje pomocou **collectAsStateWithLifecycle()**. Viewmodel sa taktiež stará o uchovávanie dát pri otočení obrazovky, preto sa dáta nenačítajú znova.

```
40  /** Obrazovka aktívneho tréningu. */
41  ± Filip Gazdik
42  @OptIn(ExperimentalMaterial3Api::class)
43  @Composable
44  fun ActiveWorkoutScreen(navController: NavController, routineId: Long, existingSessionId: Long) {
45      val context = LocalContext.current
46      val app = context.applicationContext as WorkoutApp
47
48      val vm: ActiveWorkoutViewModel = viewModel(
49          factory = ActiveWorkoutViewModel.Factory(app.repository, app.activeSessionPrefs, routineId, exist
50          key = "workout_${routineId}_${existingSessionId}"
51      )
52
53      val exercises by vm.exercises.collectAsStateWithLifecycle()
54      val workoutName by vm.workoutName.collectAsStateWithLifecycle()
55      val restTimerSeconds by vm.restTimerSeconds.collectAsStateWithLifecycle()
56      val restTimerRunning by vm.restTimerRunning.collectAsStateWithLifecycle()
57
58      val allExercises by app.repository.getAllExercises()
59          .collectAsStateWithLifecycle(initialValue = emptyList<Exercise>())
60
61      var showFinishDialog by remember { mutableStateOf( value: false) }
62      var showAddExerciseDialog by remember { mutableStateOf( value: false) }
63      var showPickExerciseFromAll by remember { mutableStateOf( value: false) }
64      var finishVolume by remember { mutableStateOf( value: 0f) }
65      var finishSessionId by remember { mutableStateOf( value: 0L) }
66
67      val notifPermLauncher = rememberLauncherForActivityResult(
68          ActivityResultContracts.RequestPermission()
```

Obr. 12: ActiveWorkoutViewModel, správa stavu aktívneho tréningu



Obr. 13: Obrázovka aktívneho tréningu

Navigácia

Navigáciu medzi obrazovkami som implementoval pomocou Navigation Compose. Celý navigačný graf aplikácie je definovaný na jednom mieste v súbore AppNavigation.kt. Každá obrazovka má svoju trasu definovanú ako reťazec, pričom parametre ako napríklad sessionId alebo routineId sú prenášané priamo v URL ceste a extrahované pri navigácii. Spätná navigácia je riešená pomocou **navController.popBackStack()**.

```

6
7
8
9  /** Navigačný graf aplikácie. */
10  @Filip Gazdik
11  @Composable
12  fun AppNavigation() {
13      val navController = rememberNavController()
14
15      // startDestination určuje, ktorá obrazovka sa zobrazí ako prvá po spustení aplikácie
16      NavHost(navController = navController, startDestination = Screen.Home.route) { this: NavGraphBuilder
17
18          // Hlavná obrazovka
19          composable(Screen.Home.route) { this: AnimatedContentScope, it: NavBackStackEntry
20              HomeScreen(navController)
21          }
22
23          // Detail/úprava rutiny
24          composable(
25              route = Screen.RoutineDetail.route,
26              arguments = listOf(navArgument("routineId") { this: NavArgumentBuilder
27                  type = NavType.LongType; defaultValue = -1L
28              })
29          ) { this: AnimatedContentScope, back ->
30              val routineId = back.arguments?.getLong(key = "routineId") ?: -1L
31              RoutineDetailScreen(navController, routineId)
32          }
33      }
34  }

```

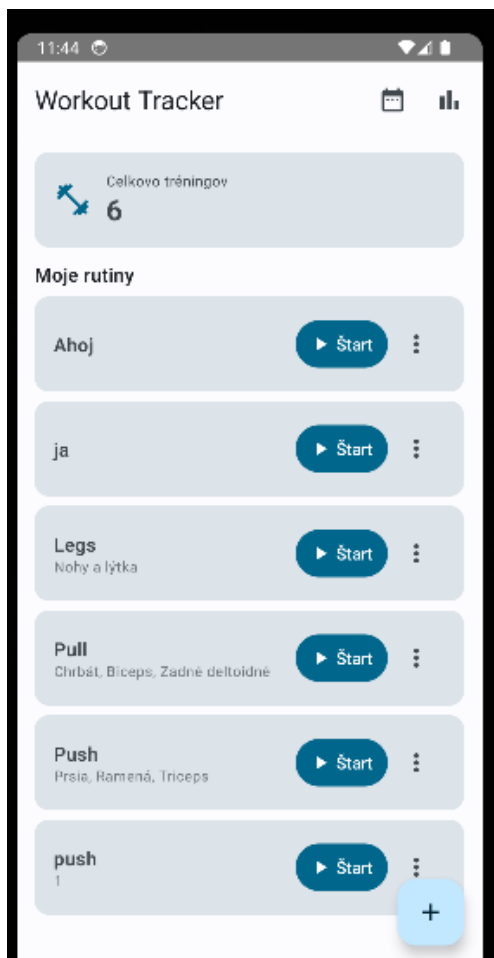
Obr. 14: Navigačný graf aplikácie

UI obrazovky

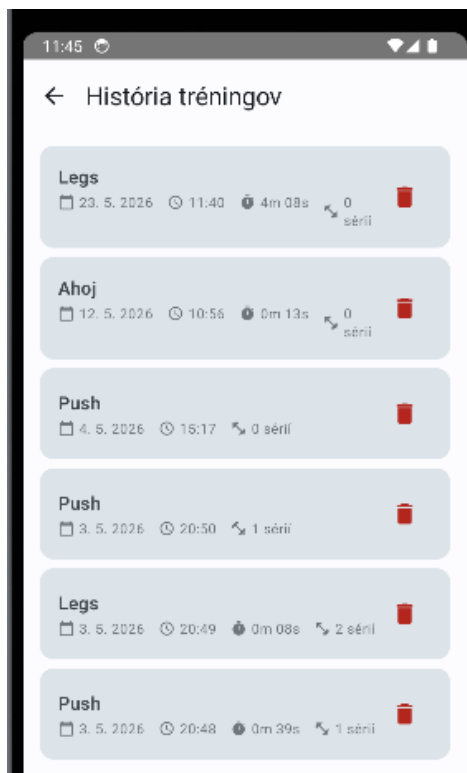
Všetky obrazovky aplikácie sú implementované ako composable funkcie pomocou Jetpack Compose. Typická obrazovka sa skladá zo Scaffold s TopAppBar a obsahovým blokom, pričom dáta získava z ViewModelu cez collectAsStateWithLifecycle(). Zoznamy sú vykresľované pomocou LazyColumn, čo zabezpečuje plynulý výkon aj pri väčšom množstve položiek.

```
1 package com.example.gazdik_vamz.ui.screens
2
3 > import ...
4
25
26 /** Obrazovka detailu dokončeného tréningu. */
27  @ Filip Gazdik
28 @OptIn(ExperimentalMaterial3Api::class)
29 @Composable
30 fun WorkoutDetailScreen(navController: NavController, sessionId: Long) {
31     val context = LocalContext.current
32     val repo = (context.applicationContext as WorkoutApp).repository
33     val vm: WorkoutDetailViewModel = viewModel(factory = WorkoutDetailViewModel.Factory(repo, sessionId))
34
35     val session by vm.session.collectAsStateWithLifecycle()
36     val sets by vm.sets.collectAsStateWithLifecycle()
37
38     val dateFormat = remember { SimpleDateFormat(pattern: "d. M. yyyy, HH:mm", Locale.getDefault()) }
39
40     // Zoskupenie sérií podľa názvu cviku
41     val grouped = remember(sets) { sets.groupBy { it.exerciseName } }
42     // Celkový objem = súčet (opakovania * váha)
43     val totalVolume = remember(sets) { sets.sumOf { (it.reps * it.weightKg).toDouble() }.toFloat() }
44
45     Scaffold(
```

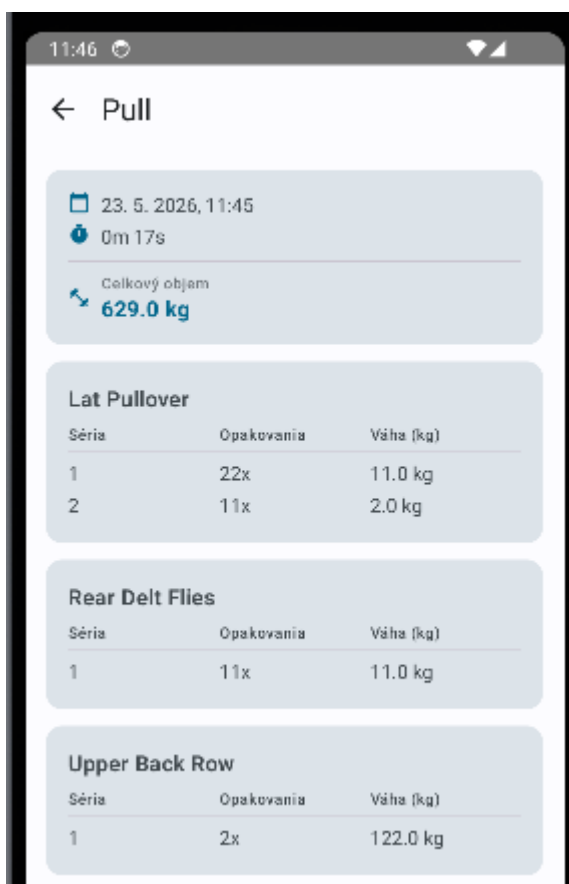
Obr. 15: Implementácia obrazovky WorkoutDetailScreen



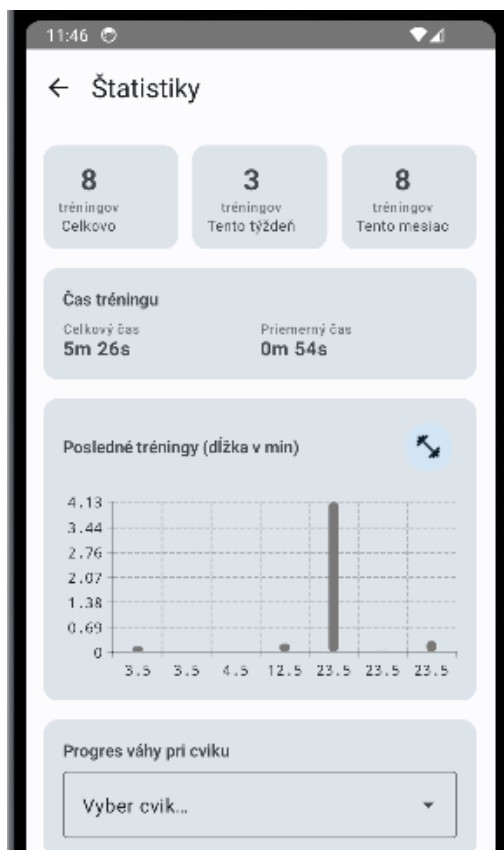
Obr. 16: Domovská obrazovka



Obr. 17: História tréningov



Obr. 18: Detail tréningu



Obr. 19: Štatistiky

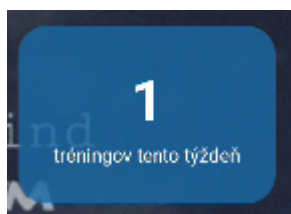
WorkManager a Widget

Aplikácia obsahuje dva komponenty bežiacie na pozadí. Prvým je WorkManager, ktorý pravidelne kontroluje dátum posledného tréningu. Ak používateľ netrénovať viac ako tri dni, systém mu automaticky odošle notifikáciu s pripomienkou. Worker je naplánovaný pri každom spustení aplikácie s periodicitou 24 hodín.

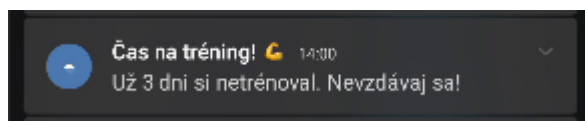
Druhým komponentom je domovský widget implementovaný pomocou AppWidgetProvider. Widget zobrazuje počet tréningov za posledných 7 dní priamo na ploche zariadenia a aktualizuje sa automaticky po každom dokončenom tréningu. Na uchovanie informácie o aktívnom tréningu medzi spusteniami aplikácie som použil SharedPreferences.

```
1 package com.example.gazdik_vamz.worker
2
3 > import ...
18
19
20 class WorkoutReminderWorker(
21     private val context: Context,
22     params: WorkerParameters
23 ) : CoroutineWorker(context, params) {
24
25
26     override suspend fun doWork(): Result {
27         val repository = WorkoutRepository(AppDatabase.getInstance(context))
28         val threeDaysAgo = System.currentTimeMillis() - TimeUnit.DAYS.toMillis( duration: 3)
29         val recentCount = repository.getSessionCountSince(threeDaysAgo)
30
```

Obr. 20: Implementácia WorkoutReminderWorker



Obr. 21: Widget na domovskej obrazovke



Obr. 22: Notifikácia pripomienky tréningu



Zdroje

ANDROID DEVELOPERS. *Jetpack Compose* [online]. Google, 2024 [cit. 2026-05-23]. Dostupné z: <https://developer.android.com/compose>

ANDROID DEVELOPERS. *Room Persistence Library* [online]. Google, 2024 [cit. 2026-05-23]. Dostupné z: <https://developer.android.com/training/data-storage/room>

ANDROID DEVELOPERS. *Navigation Compose* [online]. Google, 2024 [cit. 2026-05-23]. Dostupné z: <https://developer.android.com/develop/ui/compose/navigation>

ANDROID DEVELOPERS. *WorkManager* [online]. Google, 2024 [cit. 2026-05-23]. Dostupné z: <https://developer.android.com/topic/libraries/architecture/workmanager>

ANDROID DEVELOPERS. *App Widgets* [online]. Google, 2024 [cit. 2026-05-23]. Dostupné z: <https://developer.android.com/develop/ui/views/appwidgets>

JETBRAINS. *Kotlin Documentation* [online]. JetBrains, 2024 [cit. 2026-05-23]. Dostupné z: <https://kotlinlang.org/docs/home.html>

PATRYKANDPATRYK. *Vico — Chart library for Android* [online]. GitHub, 2024 [cit. 2026-05-23]. Dostupné z: <https://github.com/patrykandpatryk/vico>

HEVY. *Hevy — Workout Tracker* [online]. Google Play, 2024 [cit. 2026-05-23]. Dostupné z: <https://play.google.com/store/apps/details?id=com.hevyapp.hevy>

STRONG. *Strong — Workout Tracker* [online]. Google Play, 2024 [cit. 2026-05-23]. Dostupné z: <https://play.google.com/store/apps/details?id=io.strongapp.strong>

LIFTOFF. *Liftoff — Workout Tracker* [online]. Google Play, 2024 [cit. 2026-05-23]. Dostupné z: <https://play.google.com/store/apps/details?id=com.liftoffapp.liftoff>

ANDROID DEVELOPERS. *Guide to App Architecture* [online]. Google, 2024 [cit. 2026-05-23]. Dostupné z: <https://developer.android.com/topic/architecture>

ANDROID DEVELOPERS. *ViewModel Overview* [online]. Google, 2024 [cit. 2026-05-23]. Dostupné z: <https://developer.android.com/topic/libraries/architecture/viewmodel>

JETBRAINS. *Coroutines Guide* [online]. JetBrains, 2024 [cit. 2026-05-23]. Dostupné z: <https://kotlinlang.org/docs/coroutines-guide.html>

ANDROID DEVELOPERS. *Kotlin Flow* [online]. Google, 2024 [cit. 2026-05-23]. Dostupné z: <https://developer.android.com/kotlin/flow>

ANDROID DEVELOPERS. *Paging 3 Library* [online]. Google, 2024 [cit. 2026-05-23]. Dostupné z: <https://developer.android.com/topic/libraries/architecture/paging/v3-overview>

GOOGLE. *Material Design 3* [online]. Google, 2024 [cit. 2026-05-23]. Dostupné z: <https://m3.material.io>

Pri tvorbe projektu bol využitý nástroj umelej inteligencie (Claude od Anthropic) ako pomôcka pri písaní komentárov v kóde, úprave čitateľnosti a formátovaní dokumentácie.